Buffer Overflow

- **When exceeding the memory size allocated for an array**
- **Why a big deal?**
  - It's the #1 technical cause of security vulnerabilities
    - #1 overall cause is social engineering / user ignorance
- **Most common form**
  - Unchecked lengths on string inputs
  - Particularly for bounded character arrays on the stack
    - sometimes referred to as stack smashing

String Library Code

- **Implementation of Unix function `gets()`**

```
/* Get string from stdin */
char *gets(char *dest)
{
    int c = getchar();
    char *p = dest;
    while (c != EOF && c != '\n') {
        *p++ = c;
        c = getchar();
    }
    *p = '\0';
    return dest;
}
```

  - No way to specify limit on number of characters to read
- **Similar problems with other library functions**
  - **`strcpy`, `strcat`**: Copy strings of arbitrary length
  - **`scanf`, `fscanf`, `sscanf`,** when given **`%s`** conversion specification

Vulnerable Buffer Code

```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

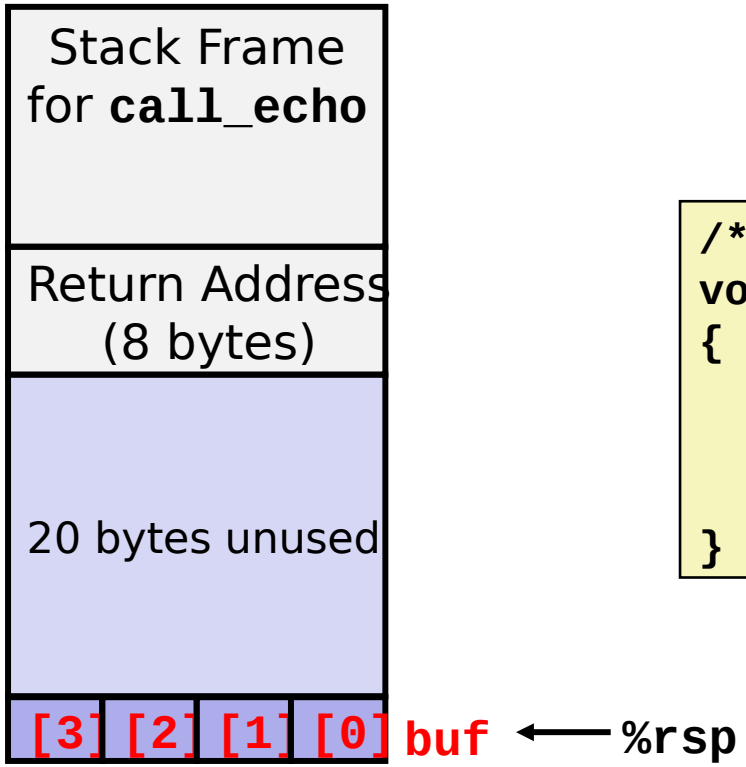
```
void call_echo() {
    echo();
}
```

```
unix>./bufdemo-nsp
Type a string:012345678901234567890123
012345678901234567890123
```

```
unix>./bufdemo-nsp
Type a string:012345678901234567890123
Segmentation Fault
```

Buffer Overflow Stack

Before call to gets




```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
  subq  $24, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

Buffer Overflow Stack Example

After call to gets

Stack Frame for **call_echo**

00	00	00	00
00	40	06	f6
00	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ⟵ %rsp

```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
   subq   $24, %rsp
   movq   %rsp, %rdi
   call   gets
   . . .
```

call_echo:

```
   . . .
   4006f1: callq  4006cf <echo>
   4006f6: add    $0x8,%rsp
   . . .
```

```
unix>./bufdemo-nsp
Type a string:012345678901234567890123
012345678901234567890123
```

```
012345678901234567890123\0
```

Overflowed buffer, but did not corrupt state

Buffer Overflow Stack Example

After call to gets

Stack Frame for **call_echo**

00	00	00	00
00	40	00	34
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ⟵ %rsp

```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
  subq  $24, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

call_echo:

```
  . . .
  4006f1: callq  4006cf <echo>
  4006f6: add    $0x8,%rsp
  . . .
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901234
Segmentation Fault
```

```
0123456789012345678901234\0
```

Overflowed buffer and corrupted return pointer

Buffer Overflow Stack Example

After call to gets

Stack Frame for **call_echo**

00	00	00	00
00	40	06	00
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ⟵ %rsp

```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
   subq   $24, %rsp
   movq   %rsp, %rdi
   call   gets
   . . .
```

call_echo:

```
   . . .
   4006f1: callq  4006cf <echo>
   4006f6: add    $0x8,%rsp
   . . .
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901230
0123456789012345678901230
```

```
0123456789012345678901230\0
```

Overflowed buffer, corrupted return pointer, but program seems to work!

Example 4

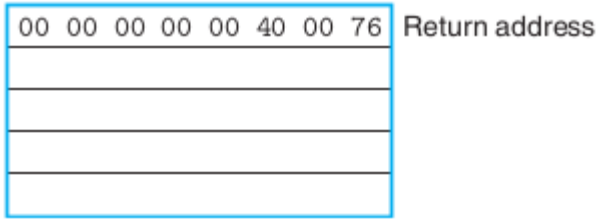
Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

(a) C code

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```



(b) Disassembly up through call to `gets`

```
      char *get_line()
1     0000000000400720 <get_line>:
2       400720:  53                   push   %rbx
3       400721:  48 83 ec 10          sub    $0x10,%rsp
      Diagram stack at this point
4       400725:  48 89 e7             mov    %rsp,%rdi
5       400728:  e8 73 ff ff ff       callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

(a) C code

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

A. Fill in the diagram that follows, indicating as much as you can about the stack just after executing the instruction at line 3 in the disassembly. Label the quantities stored on the stack (e.g., "Return address") on the right, and their hexadecimal values (if known) within the box. Each box represents 8 bytes. Indicate the position of `%rsp`. Recall that the ASCII codes for characters `0`–`9` are `0x30`–`0x39`.

(b) Disassembly up through call to `gets`

```
     char *get_line()
1    0000000000400720 <get_line>:
2      400720:  53                   push   %rbx
3      400721:  48 83 ec 10          sub    $0x10,%rsp
     Diagram stack at this point
4      400725:  48 89 e7             mov    %rsp,%rdi
5      400728:  e8 73 ff ff ff       callq  4006a0 <gets>
     Modify diagram to show stack contents at this point
```

The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

(a) C code

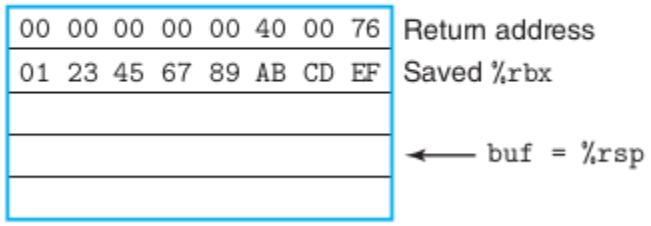
```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

(b) Disassembly up through call to `gets`

```
     char *get_line()
1    0000000000400720 <get_line>:
2      400720:  53                   push   %rbx
3      400721:  48 83 ec 10          sub    $0x10,%rsp
     Diagram stack at this point
4      400725:  48 89 e7             mov    %rsp,%rdi
5      400728:  e8 73 ff ff ff       callq  4006a0 <gets>
     Modify diagram to show stack contents at this point
```

A. Fill in the diagram that follows, indicating as much as you can about the stack just after executing the instruction at line 3 in the disassembly. Label the quantities stored on the stack (e.g., "Return address") on the right, and their hexadecimal values (if known) within the box. Each box represents 8 bytes. Indicate the position of `%rsp`. Recall that the ASCII codes for characters 0–9 are 0x30–0x39.

Stack after line 3:



The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

(a) C code

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

B. Modify your diagram to show the effect of the call to `gets` (line 5).

(b) Disassembly up through call to `gets`

```
      char *get_line()
1     0000000000400720 <get_line>:
2       400720:  53                   push   %rbx
3       400721:  48 83 ec 10          sub    $0x10,%rsp
      Diagram stack at this point
4       400725:  48 89 e7             mov    %rsp,%rdi
5       400728:  e8 73 ff ff ff       callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

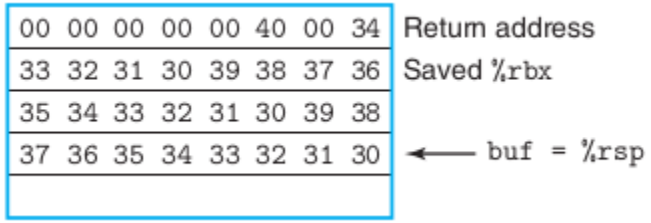
(a) C code

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

(b) Disassembly up through call to `gets`

```
      char *get_line()
1     0000000000400720 <get_line>:
2       400720:  53                   push   %rbx
3       400721:  48 83 ec 10          sub    $0x10,%rsp
      Diagram stack at this point
4       400725:  48 89 e7             mov    %rsp,%rdi
5       400728:  e8 73 ff ff ff       callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

B. Modify your diagram to show the effect of the call to `gets` (line 5).



The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

(a) C code

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

C. To what address does the program attempt to return?

(b) Disassembly up through call to `gets`

```
      char *get_line()
1     0000000000400720 <get_line>:
2       400720:  53                   push   %rbx
3       400721:  48 83 ec 10          sub    $0x10,%rsp
      Diagram stack at this point
4       400725:  48 89 e7             mov    %rsp,%rdi
5       400728:  e8 73 ff ff ff       callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

(a) C code

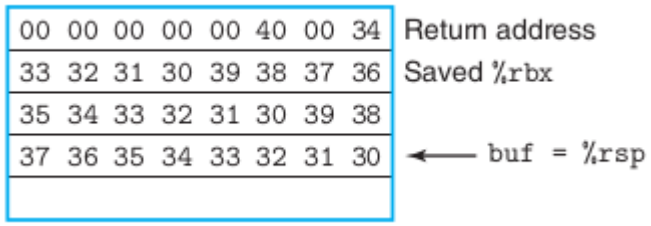
```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

(b) Disassembly up through call to `gets`

```
      char *get_line()
1     0000000000400720 <get_line>:
2       400720:  53                   push   %rbx
3       400721:  48 83 ec 10          sub    $0x10,%rsp
      Diagram stack at this point
4       400725:  48 89 e7             mov    %rsp,%rdi
5       400728:  e8 73 ff ff ff       callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

C. To what address does the program attempt to return?

The program is attempting to return to address `0x040034`. The low-order 2 bytes were overwritten by the code for character '4' and the terminating null character.



The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

Example 5

Stack discipline. Consider the following C code and its corresponding 32-bit x86 machine code. Please complete the stack diagram on the following page.

```
int bar (int a, int b) {
    return a + b;
}

int foo(int n, int m, int c) {
    c += bar(m, n);
    return c;
}
```

```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

Example 5

Draw a detailed picture of the stack, starting with the caller invoking `foo(3, 4, 5)`, and ending immediately **before** execution of the `ret` instruction in `bar`.

- The stack diagram should begin with the three arguments for `foo` that the caller has placed on the stack. To help you get started, we have given you the first one.
- Use the actual values for function arguments, rather than variable names. For example, use 3 or 4 instead of `n` or `m`.
- Always label `%ebp` and give its value when it is pushed to the stack, e.g., `%ebp: 0xffff1400`.
- You may not need to fill in all of the boxes in the diagram.

```
Value of %ebp when foo is called: 0xffffd858
Return address in function that called foo: 0x080483c9

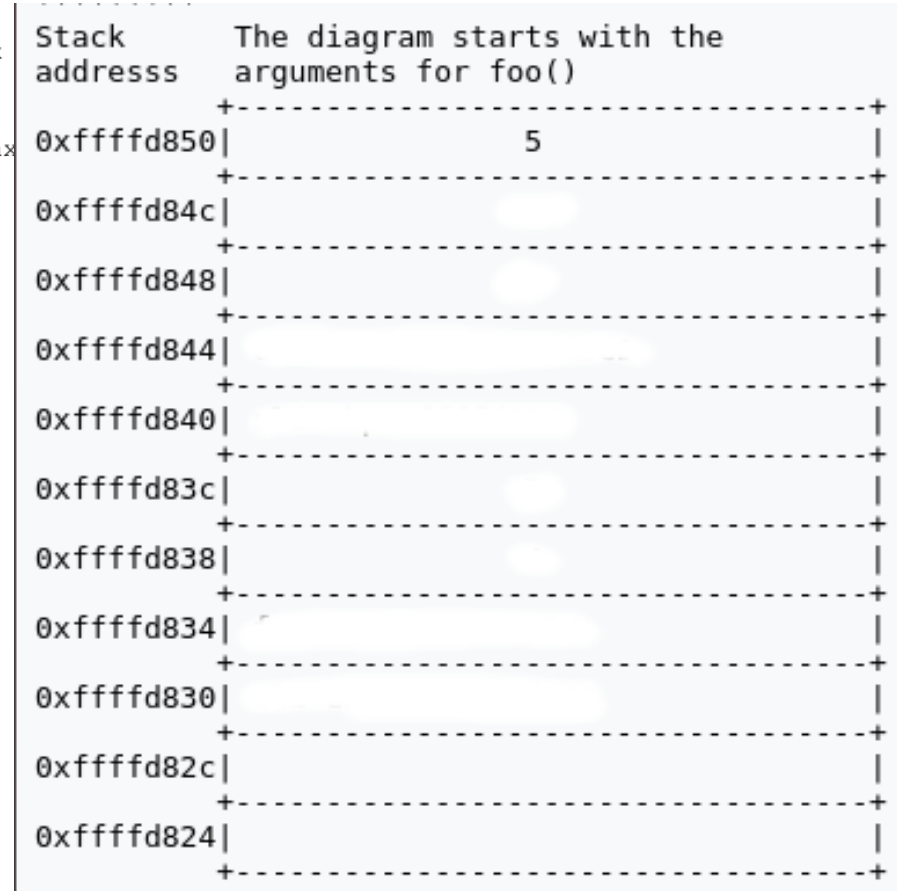
Stack       The diagram starts with the
addresss    arguments for foo()
            +-----------------------------------+
0xffffd850  |                5                  |
            +-----------------------------------+
0xffffd84c  |                                   |
            +-----------------------------------+
0xffffd848  |                                   |
            +-----------------------------------+
0xffffd844  |                                   |
            +-----------------------------------+
0xffffd840  |                                   |
            +-----------------------------------+
0xffffd83c  |                                   |
            +-----------------------------------+
0xffffd838  |                                   |
            +-----------------------------------+
0xffffd834  |                                   |
            +-----------------------------------+
0xffffd830  |                                   |
            +-----------------------------------+
```

```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd858
Return address: 0x080483c9

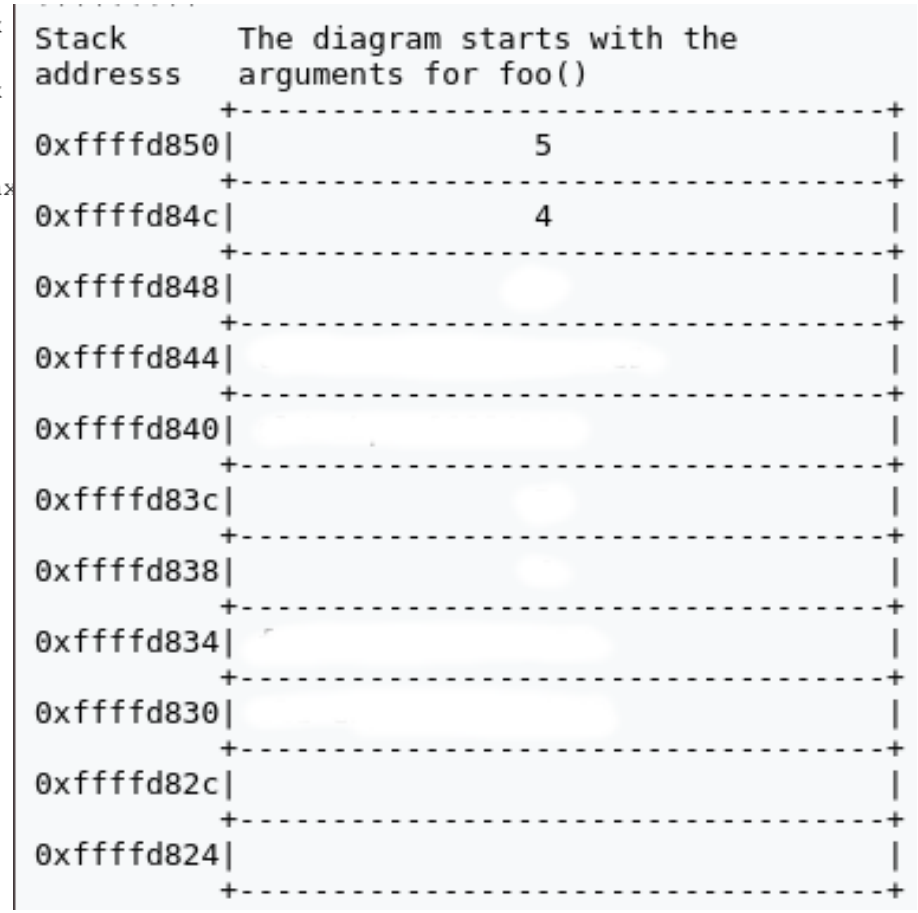


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd858
Return address: 0x080483c9

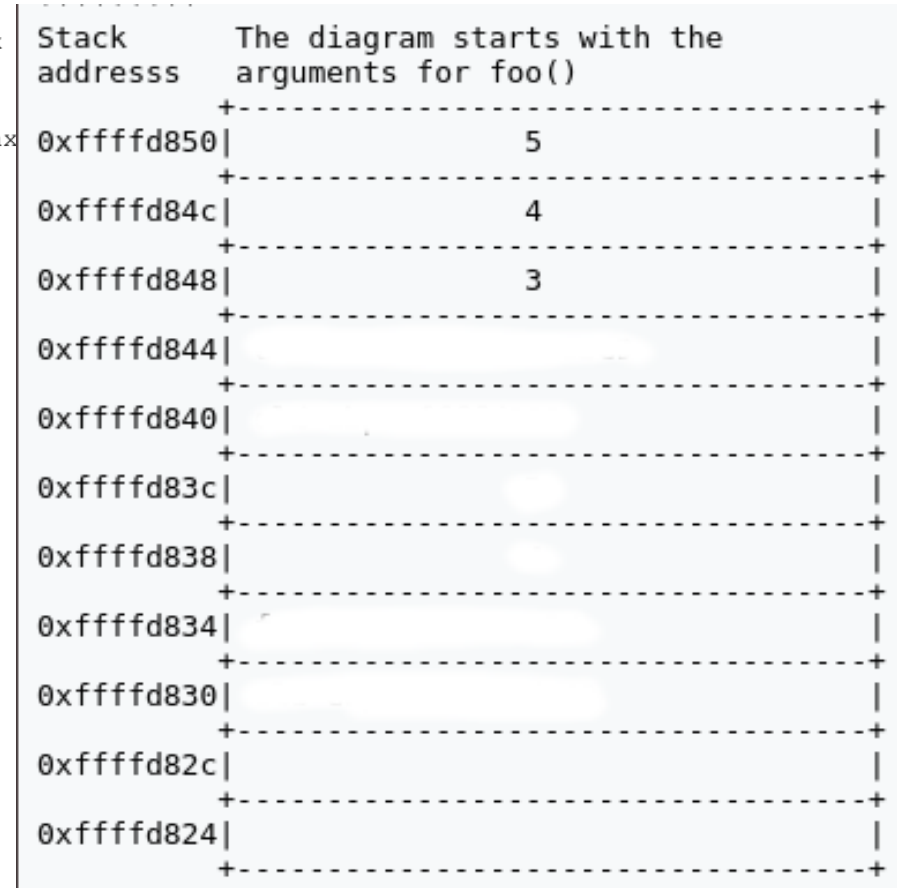


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd858
Return address: 0x080483c9

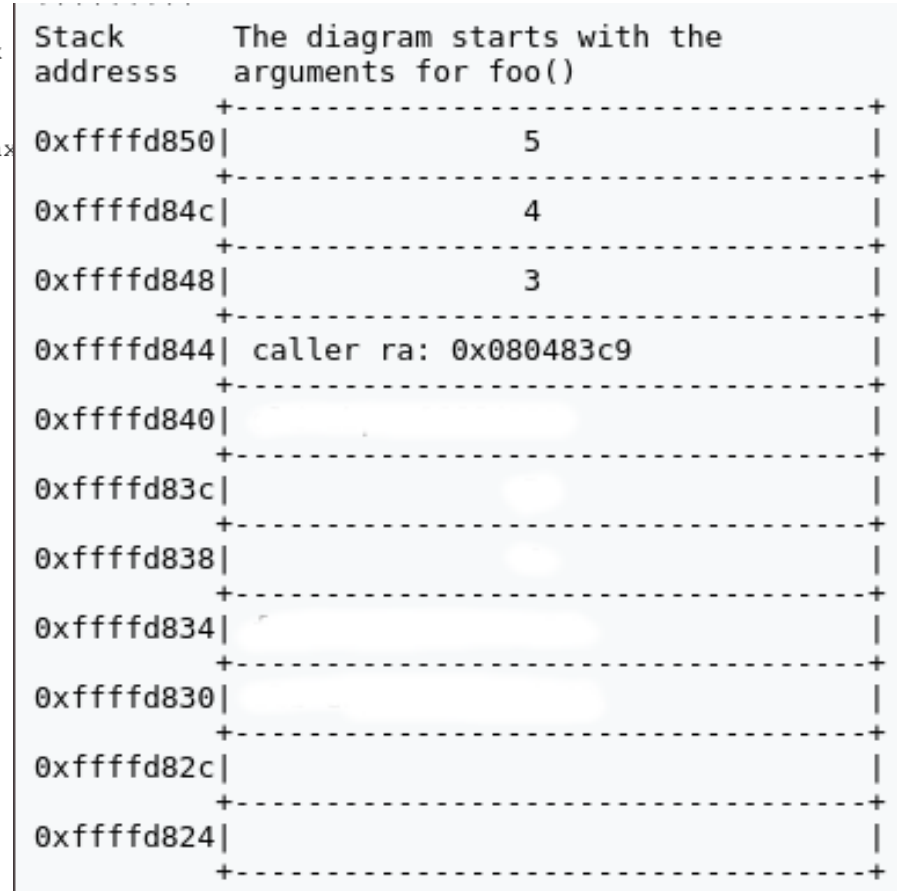


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd858
Return address: 0x080483c9

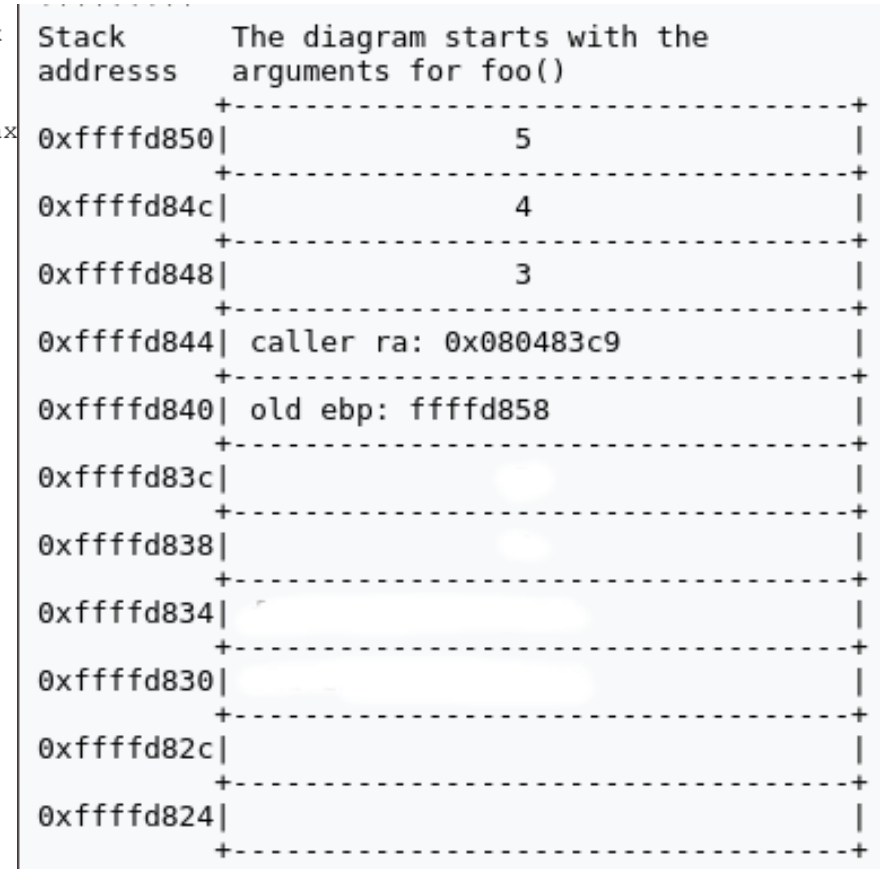


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd858
Return address: 0x080483c9
esp = 0xffffd840

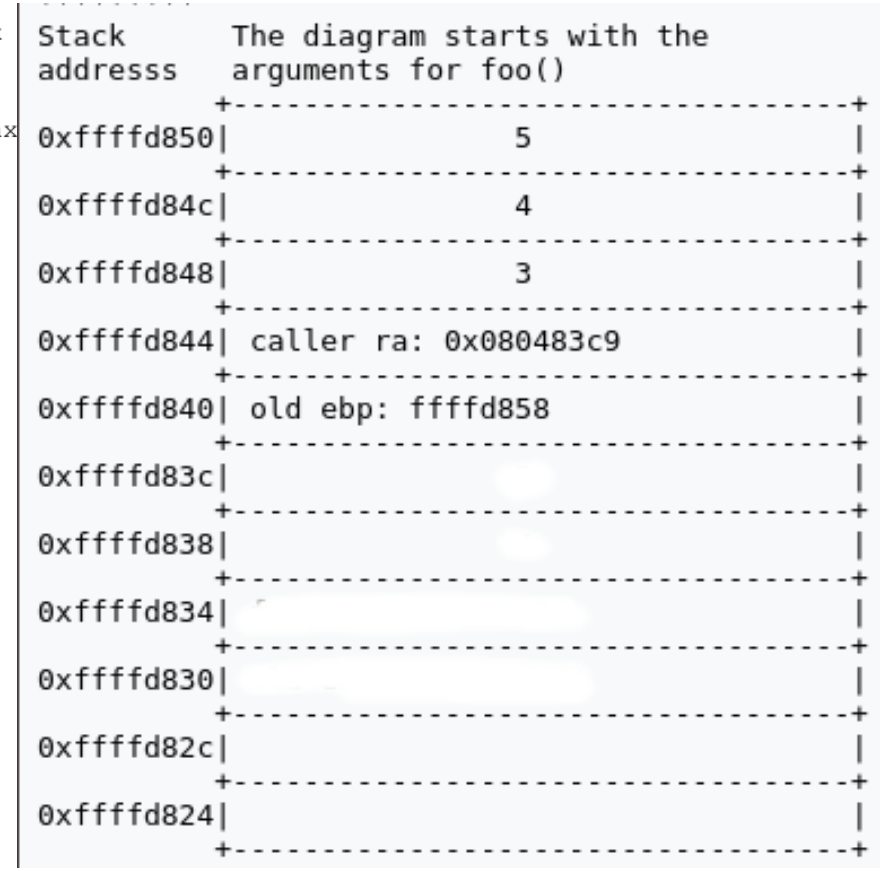


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd840

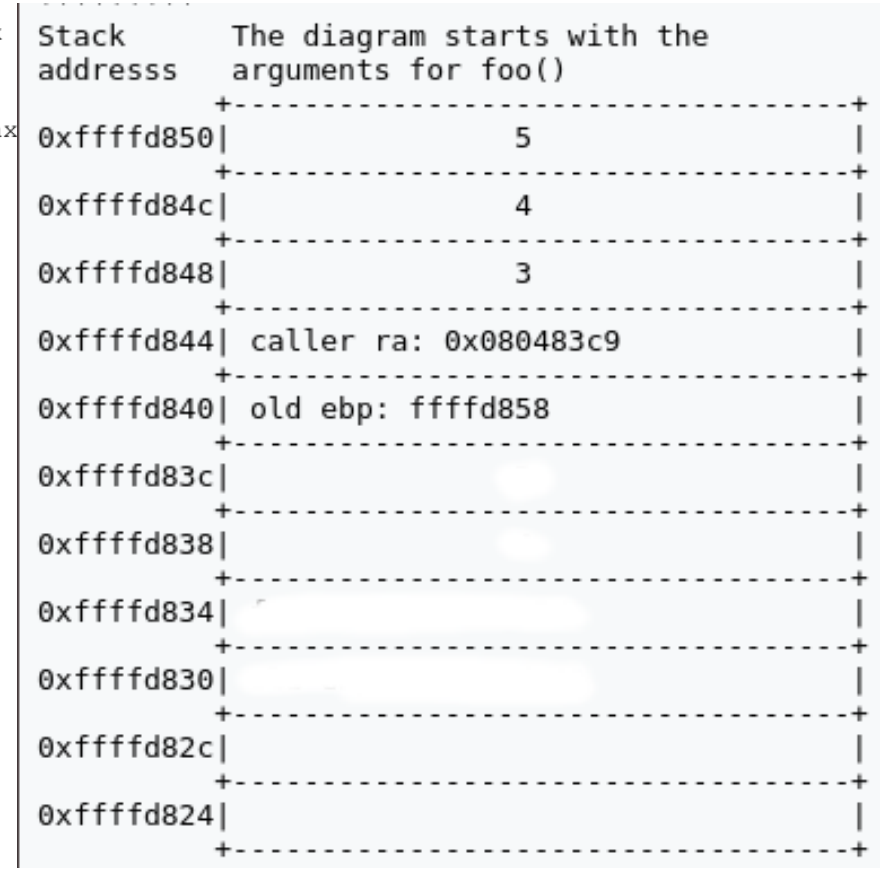


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd838

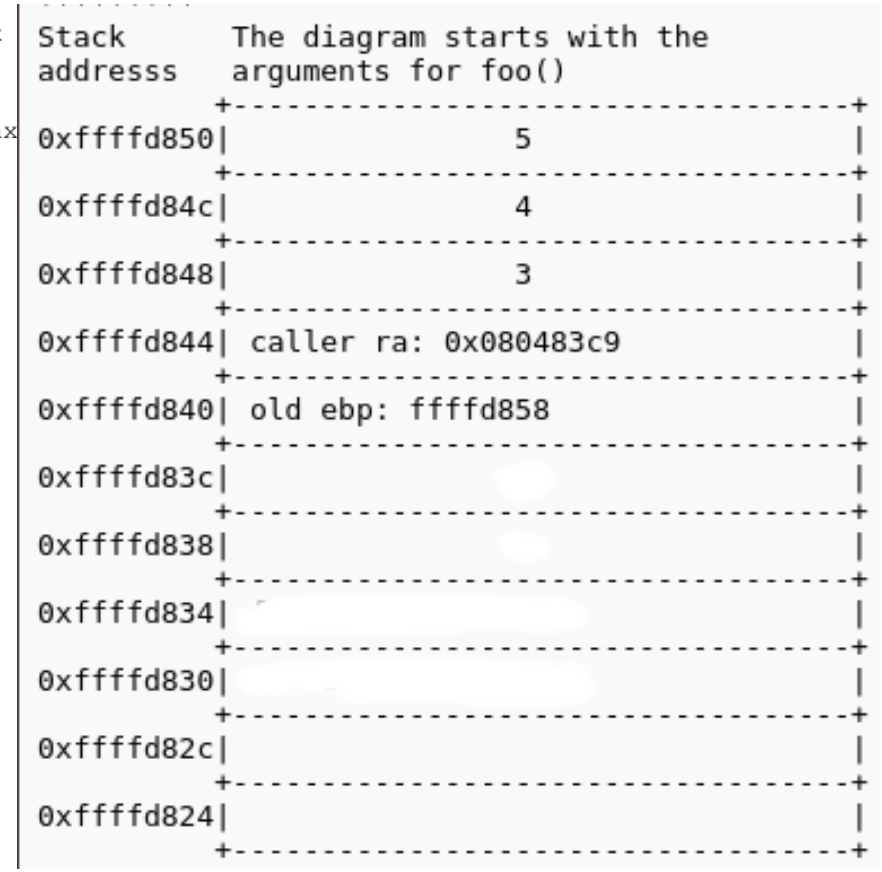


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd838
eax = 3

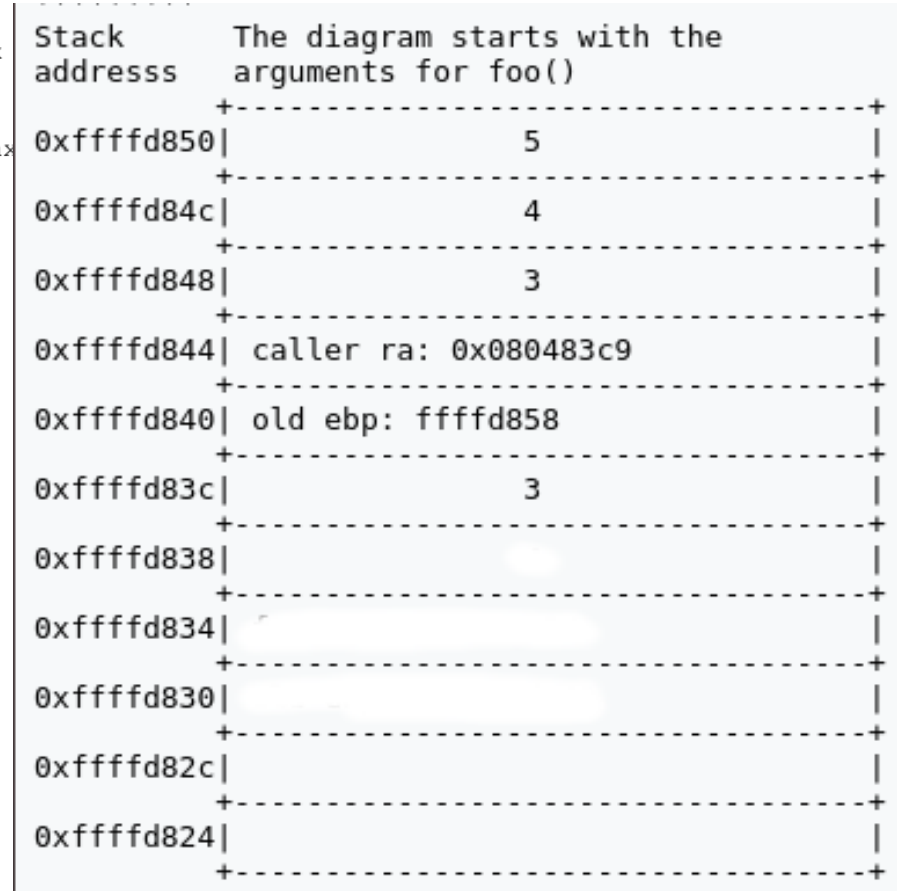


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd838
eax = 3

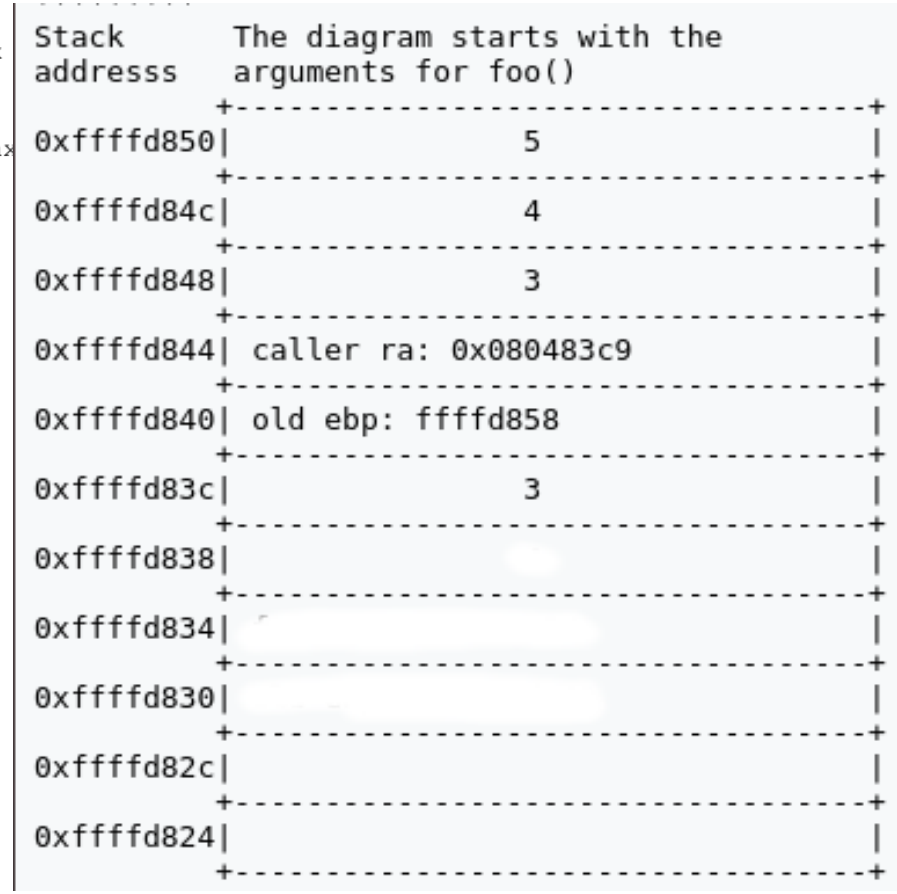


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd838
eax = 4

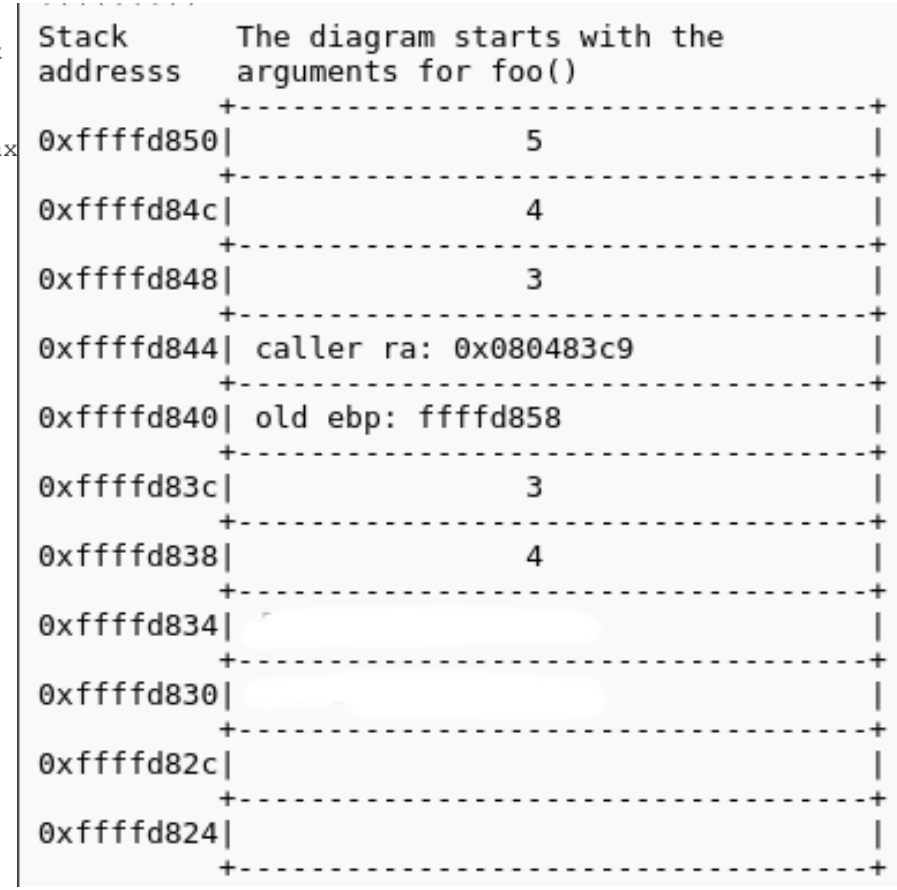


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd838
eax = 4

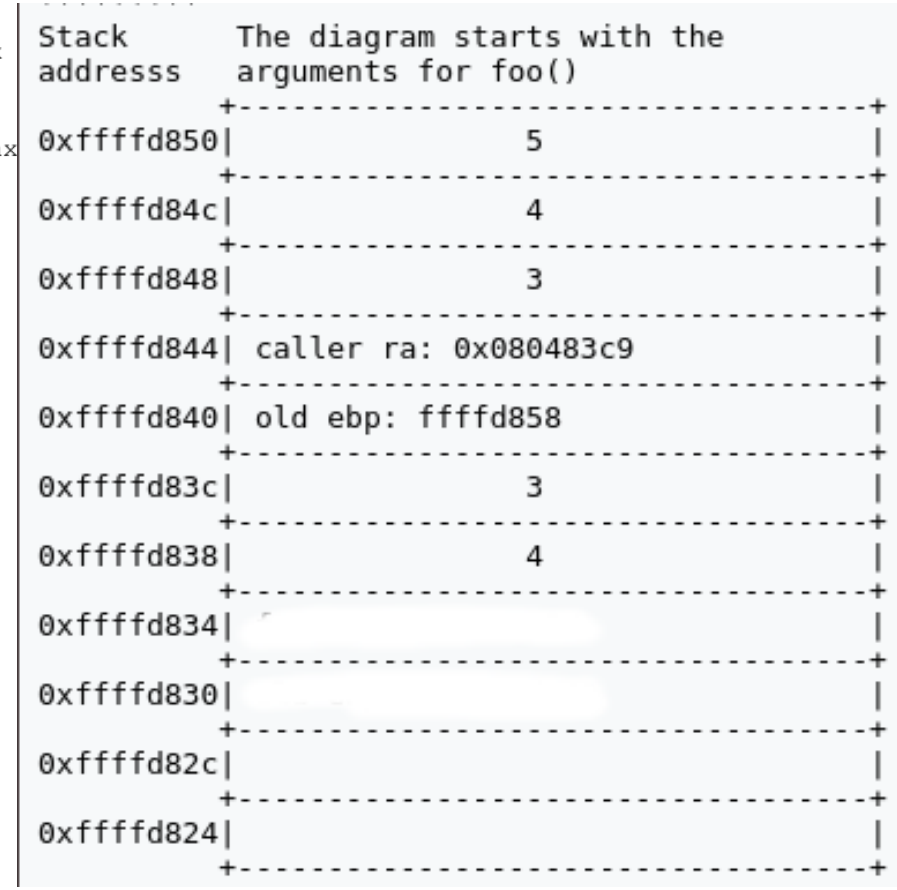


```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5) -> bar(4,3)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd838
eax = 4
Return address: 0x8048397



```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5) -> bar(4,3)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd834
eax = 4
Return address: 0x8048397

